TEK-UP University

Cycle ingénieur — ING-5-SSIR
Module : DevSecOps

RAPPORT DE PROJET

Sécurisation de bout en bout d'une application web par une démarche DevSecOps

EduGrade — portail de gestion de notes : de la version volontairement vulnérable au déploiement Kubernetes signé, observé et défendu

RÉALISÉ PAR

Amir Ghdiri
Arij Cheikhrouhou
Oumaima Ben Othmen
Nasrallah Hadhri

ENCADRÉ PAR

M. Malek Ferchichi

Année universitaire **2026 – 2027**

Novembre 2026

Résumé

Ce projet, réalisé dans le cadre du module DevSecOps, démontre qu'une chaîne de livraison logicielle peut **empêcher** une faille connue d'atteindre la production, puis **détecter** une attaque contre l'application déployée. Nous avons développé EduGrade, un portail de gestion de notes universitaires (Flask, PostgreSQL), publié d'abord dans une version **volontairement vulnérable** contenant sept failles : contrôle d'accès défaillant (IDOR), injection SQL, secrets écrits dans le code, dépendance vulnérable, NetworkPolicy permissive, en-têtes de sécurité absents et XSS réfléchi.

Un pipeline GitHub Actions de neuf contrôles (tests, SAST, détection de secrets, analyse des dépendances, de l'infrastructure et de l'image, DAST) a bloqué chacune de ces failles. Elles ont ensuite été corrigées une à une, chaque correction étant prouvée par un nouveau passage du pipeline. L'image livrée est accompagnée d'une SBOM et signée sans clé par Cosign ; le cluster k3s, entièrement décrit en Terraform, n'admet que cette image signée, impose des conteneurs non-root et isole les composants par des NetworkPolicies. Enfin, une attaque fictive a été bloquée par l'application, retrouvée dans Loki et signalée par trois alertes Grafana.

La démarche a également mis au jour des défaillances réelles — un outil de sécurité exposant des secrets dans ses journaux, des règles d'admission et des NetworkPolicies installées mais inopérantes — qui illustrent l'enseignement central du projet : **une mesure de sécurité n'existe que si un test prouve qu'elle résiste à une tentative de contournement**.

Mots-clés : DevSecOps, CI/CD, GitHub Actions, SAST, DAST, OWASP ZAP, Trivy, Gitleaks, SBOM, Cosign, Terraform, Kubernetes, k3s, Kyverno, NetworkPolicy, Grafana, Loki, STRIDE.

Sommaire

Liste des abréviations 3

Introduction générale 4

Chapitre 1 — Présentation du projet 6

1.1 L'application EduGrade 6

1.2 Architecture et technologies 6

1.3 Organisation du travail 7

Chapitre 2 — Infrastructure as Code 8

Chapitre 3 — Les vulnérabilités volontaires 10

Chapitre 4 — Le pipeline CI/CD bloquant 12

4.1 Les neuf contrôles 12

4.2 Politique de blocage 12

4.3 Premier run sur la version vulnérable 13

Chapitre 5 — Correction et vérification des failles 15

Chapitre 6 — Chaîne d'approvisionnement logicielle 18

Chapitre 7 — Déploiement sécurisé sur Kubernetes 19

Chapitre 8 — Observabilité et réponse à incident 21

Chapitre 9 — Analyse des risques (STRIDE) 23

Chapitre 10 — Difficultés rencontrées et enseignements 24

Conclusion générale et perspectives 25

Déclaration d'usage de l'intelligence artificielle 26

Annexes 27

Liste des abréviations

Sigle	Signification
CI/CD	Intégration et livraison continues (Continuous Integration / Continuous Delivery)
CSP	Content Security Policy — politique de sécurité du contenu (en-tête HTTP)
CSRF	Cross-Site Request Forgery — falsification de requête intersite
CVE	Common Vulnerabilities and Exposures — identifiant public de vulnérabilité
DAST	Dynamic Application Security Testing — test de sécurité de l'application en fonctionnement
GHCR	GitHub Container Registry — registre d'images de GitHub
IaC	Infrastructure as Code — infrastructure décrite par du code
IDOR	Insecure Direct Object Reference — accès direct non contrôlé à un objet
OIDC	OpenID Connect — protocole d'identité utilisé pour la signature sans clé
OWASP	Open Worldwide Application Security Project
PR	Pull Request — demande de fusion relue
PSA	Pod Security Admission — contrôle natif de la sécurité des pods Kubernetes
SAST	Static Application Security Testing — analyse statique du code
SBOM	Software Bill of Materials — inventaire des composants d'un logiciel
STRIDE	Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
XSS	Cross-Site Scripting — injection de script dans une page

> **Convention de preuve**
>
> Chaque résultat cité renvoie à une preuve numérotée du dépôt (par exemple *preuve 41*) : un journal contenant la commande exacte, sa sortie complète et son code de retour, accompagné de sa capture. Preuves 00 à 34 : Partie 1 (`docs/preuves/`) ; preuves 40 à 105 : Partie 2 (`docs/preuves/corrvulnecicd/`). Aucune valeur de secret n'y figure.

INTRODUCTION GÉNÉRALE

Contexte, problématique et objectifs

Contexte

Les applications web sont aujourd'hui livrées en continu : chaque modification du code peut atteindre la production en quelques minutes. Cette rapidité multiplie les occasions d'introduire une faille. La démarche **DevSecOps** répond à ce risque en intégrant la sécurité à chaque étape du cycle de vie — écriture, construction, test, livraison, déploiement et exploitation — sous forme de contrôles automatiques plutôt que d'audits ponctuels.

Problématique

Comment garantir, de manière **vérifiable**, qu'une faille connue ne peut pas être livrée, que ce qui s'exécute en production est exactement ce qui a été contrôlé, et qu'une attaque contre l'application déployée est détectée ?

Démarche

Pour répondre à cette question de façon démontrable, nous avons volontairement introduit des failles dans une application, puis construit la chaîne qui doit les arrêter. Chaque faille suit le même cycle, documenté par des preuves : **introduire** (version `v0.1-vulnerable`) → **détecter** automatiquement → **bloquer** la livraison → **corriger** (une Pull Request par faille) → **vérifier** le retour au vert.

Objectifs

- **Tout déclarer en code** : dépôt, protection de branche, secrets, cluster, politiques de sécurité, tableaux de bord.
- **Bloquer la livraison** dès qu'une faille exploitable est détectée.
- **Garantir l'intégrité** de l'image déployée : scannée, inventoriée, signée, vérifiée à l'admission.
- **Durcir l'exécution** : conteneurs non-root, segmentation réseau, refus des images non conformes.
- **Observer et réagir** : journaux centralisés, alertes et plan de réponse à incident.

Résultats obtenus

7	9	57	12/12	3
failles détectées et corrigées	contrôles bloquants dans le pipeline	tests automatisés dont 5 de sécurité	tests de segmentation et d'admission	alertes déclenchées par l'attaque fictive

Le rapport suit l'ordre de la chaîne : présentation du projet (chapitre 1) et de son infrastructure (chapitre 2), failles introduites (chapitre 3), pipeline qui les bloque (chapitre 4) et leur correction (chapitre 5), intégrité de l'image livrée (chapitre 6), déploiement durci (chapitre 7), observabilité (chapitre 8), puis analyse des risques (chapitre 9) et retour d'expérience (chapitre 10).

CHAPITRE 1

Présentation du projet

> Ce chapitre présente l'application EduGrade, son architecture technique et l'organisation adoptée par l'équipe.

1.1 L'application EduGrade

EduGrade est un portail web de gestion de notes universitaires. Toutes les données qu'il manipule (identités, notes, secrets) sont fictives.

Rôle	Fonctionnalités
Administrateur	Gestion des comptes enseignants et étudiants : création avec mot de passe haché, modification, suppression avec confirmation.
Enseignant	Gestion des notes et des matières, recherche des étudiants, statistiques de la promotion.
Étudiant	Lecture seule : tableau de bord (moyenne, mention, meilleure note) et consultation de **ses** notes.

Mesures de sécurité de base : contrôle des rôles côté serveur, mots de passe hachés avec scrypt, jeton anti-CSRF sur tous les formulaires, cookie de session `HttpOnly` et `SameSite`, interface sans JavaScript ni ressource externe. L'application écrit une ligne JSON par événement de sécurité (`login_failed`, `access_denied`, `suspicious_input`…) sans jamais y inscrire de mot de passe.

1.2 Architecture et technologies

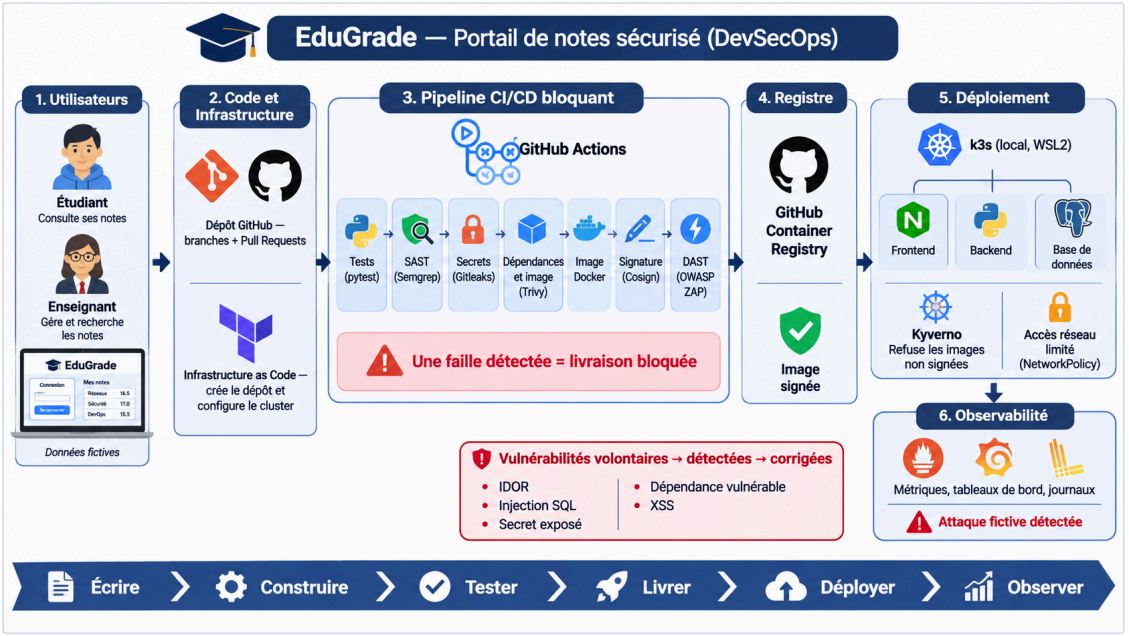


Figure 1 — Architecture d'ensemble : du code au cluster observé, en six étapes.

Couche	Technologies
Application	Python 3.12, Flask 3.1, Jinja2, psycopg 3, Gunicorn ; PostgreSQL 16
Conteneurs	Dockerfile multi-étapes, utilisateur UID 10001, système de fichiers en lecture seule ; Nginx non-root
Infrastructure as Code	Terraform 1.17 (providers `github`, `kubernetes`, `helm`, `random`)
CI/CD et sécurité	GitHub Actions, pytest, Semgrep, Bandit, Gitleaks, Trivy, Checkov, Syft, Cosign, OWASP ZAP
Orchestration	k3s 1.37 (WSL2), Traefik, Kyverno 1.20, Pod Security Admission
Observabilité	Prometheus, Alertmanager, Grafana 13, Loki 3.6, Grafana Alloy

Chemin d'une requête : `navigateur → Traefik → frontend Nginx :8080 → backend Flask :8000 → PostgreSQL :5432`. Seul Traefik joint le frontend et seul le backend joint la base (chapitre 7).

1.3 Organisation du travail

Branche	Rôle et règles
`main`	Version livrée, **protégée** : Pull Request obligatoire, approbation par un autre membre, contrôle CI `build-test` vert et à jour, ni force push ni suppression.
`develop`	Branche d'intégration : toutes les branches de travail en partent et y reviennent par Pull Request.
`feature/*`, `fix/*`	Évolutions et corrections : une faille = une branche = une Pull Request.
Tags	`v0.1-vulnerable` (avant correction), `v1.0-secure` (version finale).

Vingt-trois Pull Requests ont été ouvertes et relues. Depuis l'introduction du pipeline (PR #9), chacune déclenche les neuf contrôles de sécurité. La répartition détaillée du travail figure dans les fiches de contribution individuelles.

CHAPITRE 2

Infrastructure as Code

> Aucune ressource du projet n'a été configurée à la main : le dépôt GitHub comme le cluster sont décrits en Terraform, relus en Pull Request et reproductibles.

2.1 Le dépôt GitHub

`terraform/github/` déclare le dépôt, les branches `main` et `develop`, la protection de `main`, la collaboratrice (droit `push`, sans administration), les alertes Dependabot et les GitHub Secrets `APP_SECRET_KEY` et `POSTGRES_PASSWORD` utilisés par la CI. Leurs valeurs ne figurent que dans un fichier `terraform.tfvars` non versionné.



Figure 2 — `terraform plan` du dépôt : 8 ressources, secrets masqués *(sensitive value)* (preuve 04).

2.2 Le cluster Kubernetes

`terraform/cluster/` crée 20 ressources :

- les namespaces `edugrade` (Pod Security Admission **restricted**) et `monitoring` ;
- les Secrets Kubernetes de l'application et de Grafana, **générés aléatoirement** par Terraform et jamais écrits dans git ;
- cinq NetworkPolicies (chapitre 7) ;
- par Helm, avec des versions épinglées : Kyverno, kube-prometheus-stack, Loki et Alloy, ainsi que le tableau de bord et les alertes Grafana.

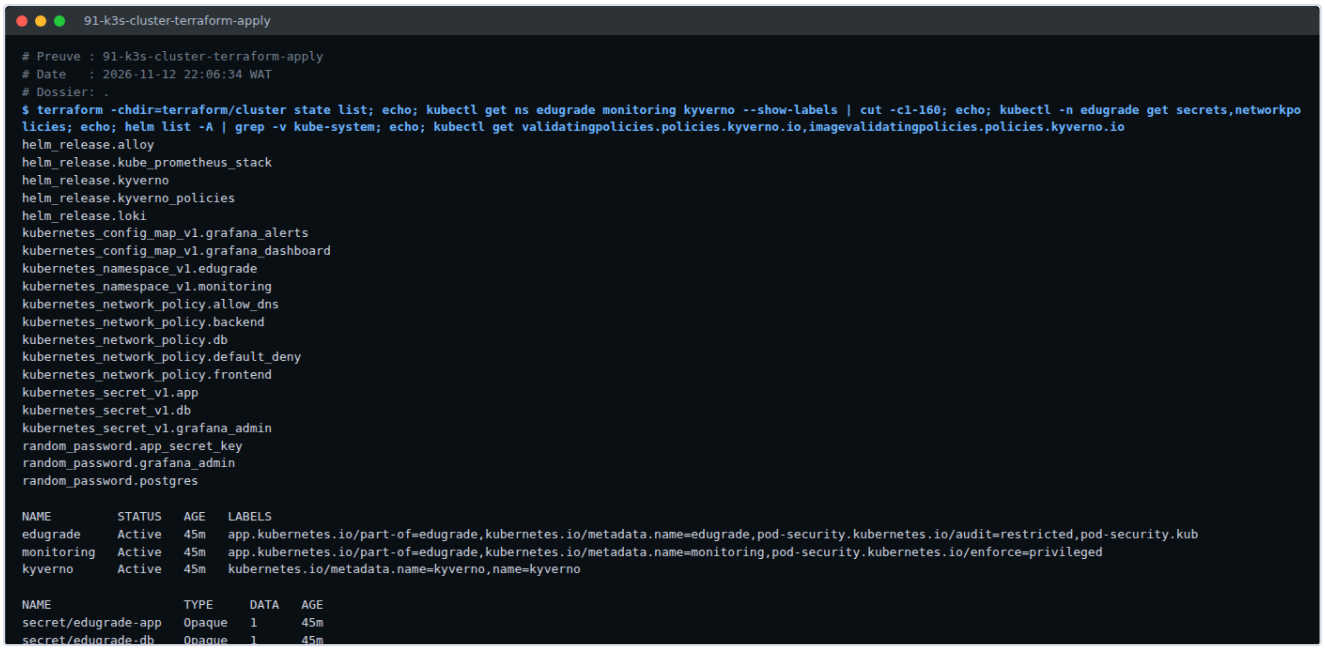


Figure 3 — Cluster appliqué : état Terraform, namespaces et labels Pod Security, Secrets, NetworkPolicies, releases Helm, règles Kyverno prêtes (preuve 91).

CHAPITRE 3

Les vulnérabilités volontaires

> Sept failles ont été introduites dans la version `v0.1-vulnerable`. Chacune est signalée dans le code par un commentaire `VULN-…` et documentée dans `docs/VULNERABILITES.md`.

ID	Faille	Emplacement	OWASP	Impact démontré
V1	IDOR sur les notes	`app/student.py`	A01	Un étudiant lit les notes et l'e-mail de n'importe quel autre étudiant.
V2	Injection SQL	`app/teacher.py`	A03	Une requête `UNION` exfiltre les 8 hachages de mots de passe.
V3	Secrets en dur	`app/config.py`	A07	Avec la clé lue dans le dépôt public, un cookie enseignant forgé donne accès sans mot de passe.
V4	PyYAML 5.3.1	`requirements.txt`	A06	CVE-2020-14343 (CRITICAL) : exécution de code par un fichier YAML.
V5	NetworkPolicy permissive	`terraform/cluster/network.tf`	A05	Le backend peut sortir vers tout Internet : exfiltration possible.
B1	En-têtes HTTP absents	`docker/nginx.conf`	A05	Ni CSP, ni protection anti-clickjacking, ni `nosniff`.
B2	XSS réfléchi	`students.html`	A03	Script exécuté dans la session d'un enseignant.

3.1 Démonstration sur l'application

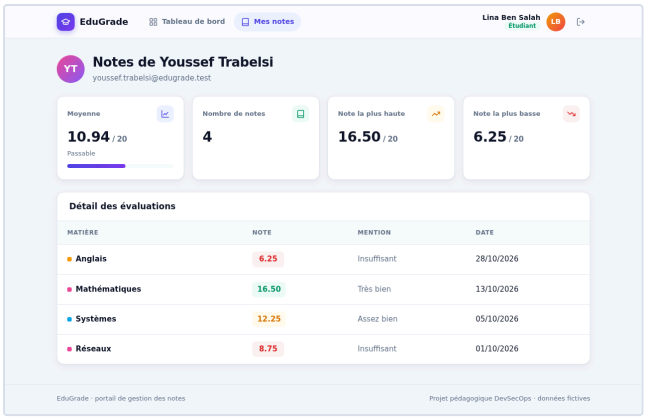


Figure 4 — V1 : connectée en Lina, la page de Youssef s'affiche (preuve 26).

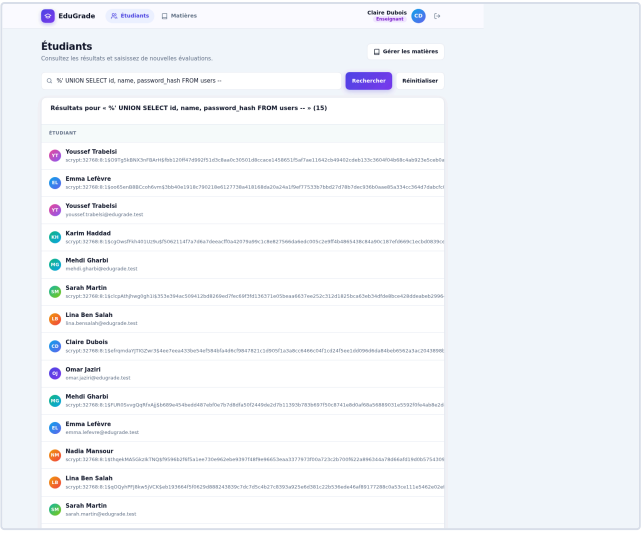


Figure 5 — V2 : les hachages scrypt apparaissent (preuve 28).

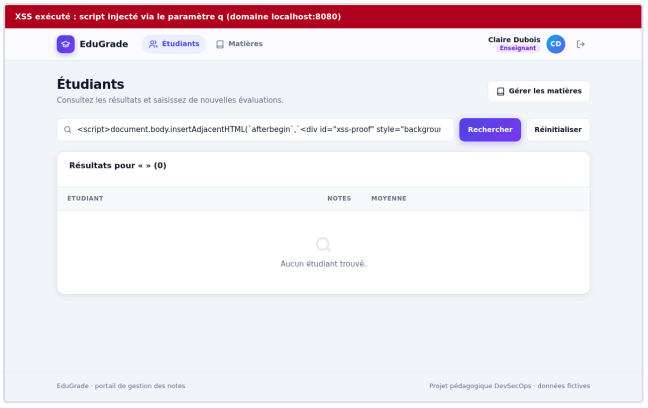


Figure 6 — Bonus 2 : le script injecté s'exécute (preuve 29).

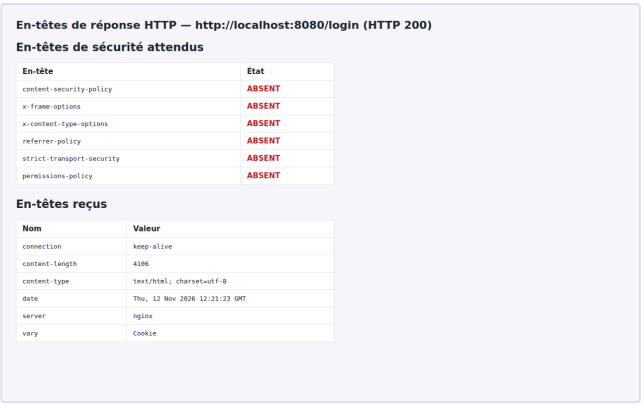


Figure 7 — Bonus 1 : aucun en-tête de sécurité (preuve 30).

CHAPITRE 4

Le pipeline CI/CD bloquant

> Le pipeline `.github/workflows/ci.yml` s'exécute sur chaque Pull Request et chaque push vers `develop` et `main`. Une faille exploitable détectée empêche la publication de l'image.

4.1 Les neuf contrôles

Job	Outils	Bloque si…
`build-test`	pytest + PostgreSQL de service	un test fonctionnel échoue (contrôle exigé par la protection de `main`)
`security-tests`	pytest `-m security`	un test d'IDOR, d'injection SQL ou de XSS échoue
`sast`	Semgrep, Bandit	constat Semgrep **ERROR** ; Bandit **HIGH**
`secrets`	Gitleaks sur tout l'historique git	une fuite, hors exceptions ciblées
`dependencies`	Trivy fs	vulnérabilité HIGH/CRITICAL **corrigeable**
`iac`	Trivy config (Checkov en rapport)	HIGH/CRITICAL hors exceptions datées
`image`	docker build, Syft, Trivy image	HIGH/CRITICAL corrigeable dans l'image
`publish`	GHCR, Cosign	s'exécute uniquement si **tout** est vert, jamais sur une PR
`dast`	Docker Compose + OWASP ZAP authentifié	au moins une alerte **High**

4.2 Politique de blocage

La politique complète (`docs/POLITIQUE_BLOCAGE.md`) repose sur quatre principes :

- **Une faille exploitable bloque, le bruit ne bloque pas.** Les seuils sont calibrés sur la version vulnérable : chaque faille est bloquée par au moins un contrôle, sans rendre le pipeline rouge en permanence (les vulnérabilités Debian sans correctif restent visibles dans la SBOM mais ne bloquent pas).
- **Ce qui est scanné est ce qui est livré.** L'image est construite une seule fois, puis testée par ZAP, publiée et signée sans reconstruction.
- **Le DAST est authentifié.** ZAP se connecte en enseignant (jeton CSRF lu dans la page) pour tester les pages réellement exposées.
- **Les exceptions sont explicites.** Déclarées dans un fichier versionné, limitées à un fichier ou à une empreinte, justifiées et datées.

4.3 Premier run sur la version vulnérable

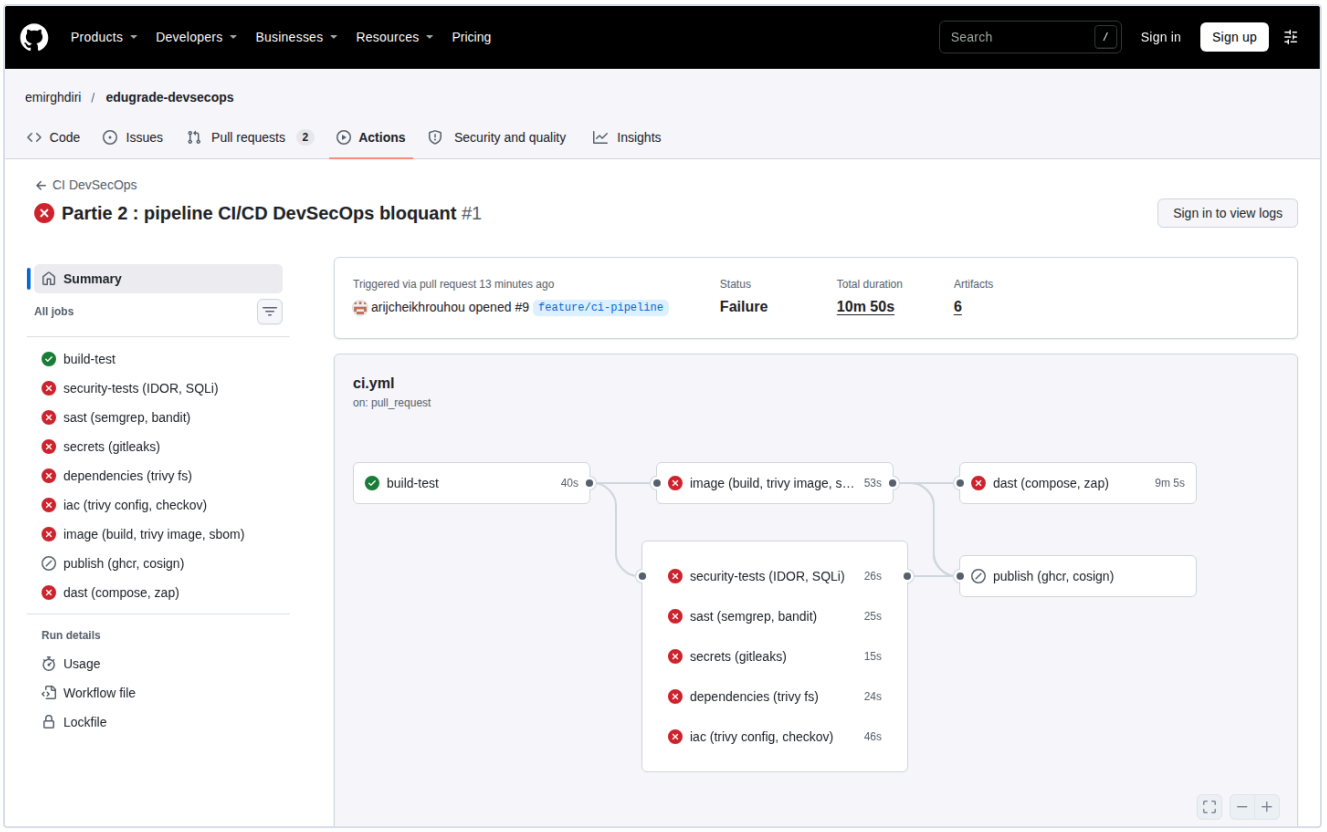


Figure 8 — Premier run : `build-test` vert, 7 contrôles rouges, `publish` sauté — aucune image vulnérable n'est publiée (preuve 40).

Job	Ce qui a été détecté	Preuve
security-tests	4 échecs : notes d'un autre étudiant accessibles, aucun `access_denied` ; tautologie SQL ; hachages divulgués	41
sast	Semgrep `sqlalchemy-execute-raw-query` (V2) et `template-unescaped-with-safe` (B2)	42
secrets	Gitleaks : 2 secrets dans `app/config.py` (V3)	43
dependencies	PyYAML 5.3.1, CVE-2020-14343 CRITICAL (V4)	44
image	V4 et **zlib vulnérable** dans l'image de base — trouvaille hors failles volontaires	45
iac	Dépôt public (CRITICAL) et commits non signés (HIGH)	46
dast	ZAP : **3 alertes High** (injection SQL, XSS réfléchi, XSS DOM) et en-têtes absents	47

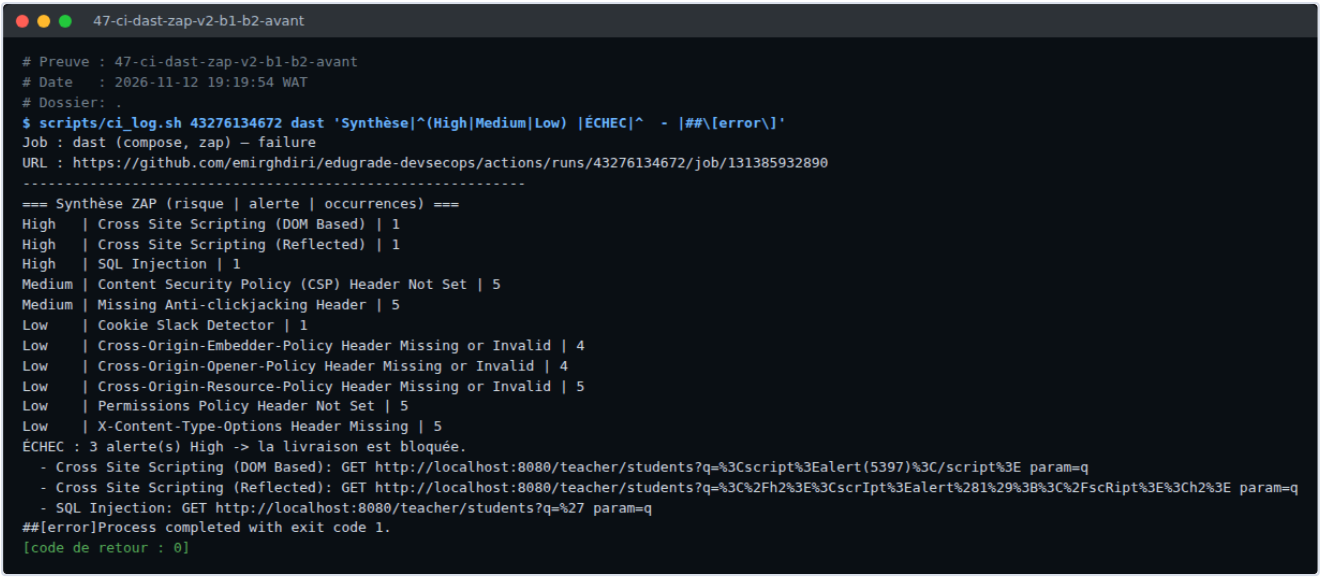


Figure 9 — Synthèse ZAP du premier run : trois alertes High, la livraison est bloquée (preuve 47).

Constat : une erreur du pipeline lui-même (preuves 48, 51, 52)

Bandit faisait échouer son job alors qu'aucun constat n'était HIGH, et surtout il **affichait en clair les secrets V3 dans le journal public** de GitHub Actions. Correction : format de sortie sans valeur, blocage sur HIGH uniquement. Un outil de sécurité mal configuré peut lui-même devenir une source de fuite.

CHAPITRE 5

Correction et vérification des failles

> Chaque faille a été corrigée dans sa propre branche `fix/*` et sa propre Pull Request. Les preuves suivent toujours le même triptyque : *erreur → correction → après*.

5.1 Synthèse des corrections

Faille	Correction appliquée	Vérification	Preuves
V1 PR #10	Contrôle d'appartenance côté serveur : réponse **403** et événement `access_denied` ; enseignants et administrateur gardent l'accès.	tests IDOR **PASSED**	41, 60, 61
V2 PR #11	Requête paramétrée (`ILIKE %s` et paramètre lié).	tests SQLi **PASSED**, SAST vert	42, 62, 63
B2 PR #12	Suppression du filtre `|safe` , test de non-régression XSS.	plus d'alerte ZAP XSS	47, 64, 65
B1 PR #13	En-têtes dans Nginx : CSP stricte, X-Frame-Options, nosniff, Referrer-Policy, Permissions-Policy, COOP/CORP/COEP.	plus d'alerte ZAP d'en-tête	47, 66, 67
V3 PR #14	Secrets lus uniquement dans l'environnement, **rotation**, exception Gitleaks limitée aux deux empreintes historiques.	`no leaks found` ; ancienne clé refusée	43, 68-70
V4 PR #15	PyYAML 6.0.3 et `yaml.safe_load` .	analyse des dépendances verte	44, 71, 72
zlib PR #16	Correctifs Debian appliqués à chaque construction de l'image.	plus de HIGH dans l'image	45, 73, 74
IaC PR #17	Choix documentés : exceptions limitées à un fichier, expirant le 30/06/2027.	analyse IaC verte	46, 75, 76

5.2 Exemples : V1 et V3

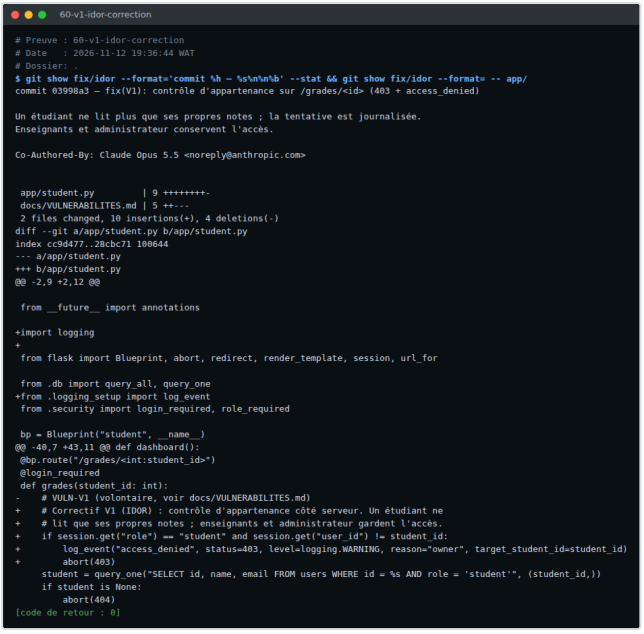


Figure 10 — Correction V1 : contrôle d'appartenance (preuve 60).

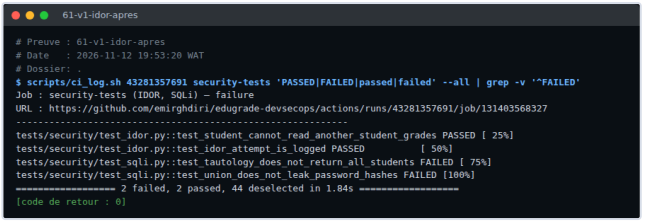


Figure 11 — Après : les tests IDOR passent en CI (preuve 61).



Figure 12 — V3 : un cookie forgé avec l'ancienne clé est désormais refusé (redirection vers `/login`) : la rotation est effective (preuve 70).

5.3 Résultat : le pipeline est vert

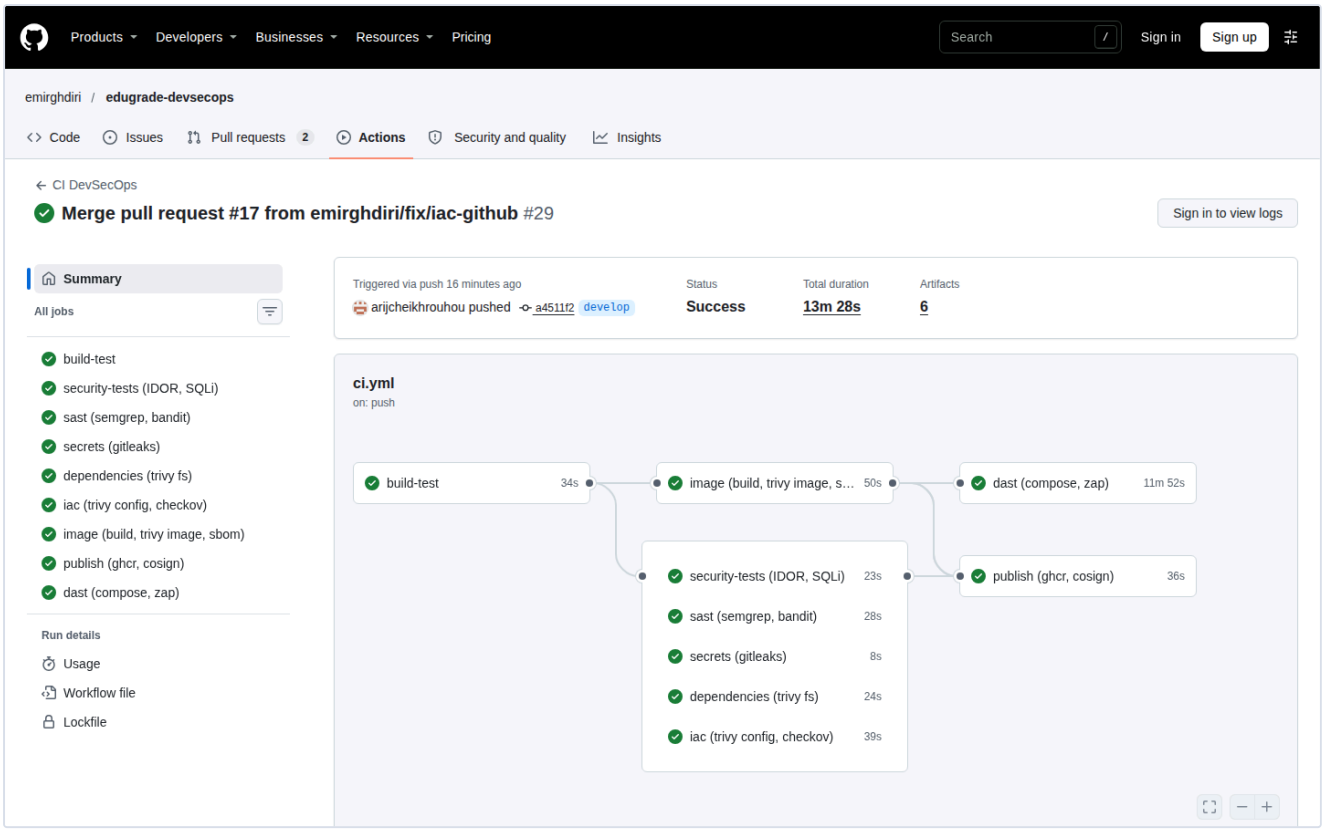


Figure 13 — Pipeline de `develop` après fusion des corrections : 9 jobs sur 9 verts, image publiée et signée (preuves 85-86).

Bilan

Avant : 7 contrôles rouges et publication bloquée. Après : 9 contrôles verts, ZAP sans alerte High, image publiée et signée. Chaque transition est tracée par une Pull Request relue et un run consultable sur GitHub.

CHAPITRE 6

Chaîne d'approvisionnement logicielle

> Le cluster doit pouvoir vérifier que l'image qu'il exécute est bien celle que le pipeline a contrôlée. Le job `publish` le permet ; il ne s'exécute que si les sept contrôles précédents sont verts.

6.1 Étapes de publication

1. **Publication** sur GHCR avec pour tag le SHA du commit, puis lecture du **digest** immuable de l'image.
2. **Signature sans clé** par Cosign : le certificat est émis pour l'identité OIDC du workflow (`…/ci.yml@refs/heads/develop`) et la signature est inscrite dans le journal de transparence public **Rekor**.
3. **Attestation de la SBOM** SPDX produite par Syft (101 composants pour l'image finale, preuve 83).
4. **Vérification** de la signature avant la fin du job.

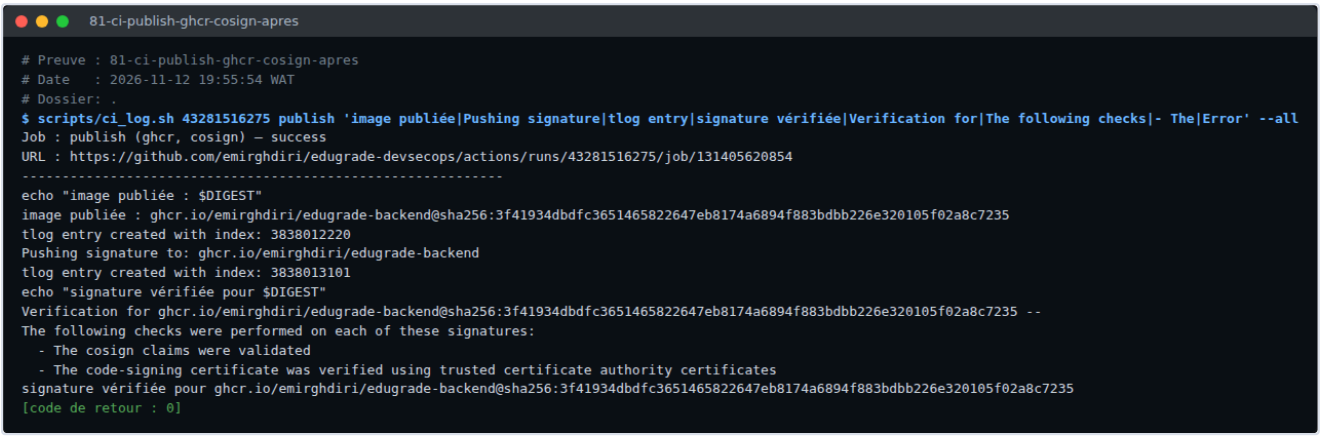


Figure 14 — Publication sur GHCR, signature Cosign, entrées Rekor et vérification (preuve 81).

6.2 Intérêt

Aucune clé privée n'est à protéger : l'identité de signature est celle du pipeline lui-même. C'est cette identité que Kyverno exige à l'admission (chapitre 7) : une image construite à la main, ou signée depuis une autre branche, est refusée. La SBOM permet, à l'annonce d'une nouvelle CVE, de savoir immédiatement si l'image déployée est concernée.

CHAPITRE 7

Déploiement sécurisé sur Kubernetes

> L'application tourne sur un cluster k3s local. Quatre couches de protection se complètent ; la cinquième faille volontaire (V5) y a été introduite puis corrigée.

7.1 Défense en profondeur

Couche	Mesures
Pod Security Admission	Namespace en **restricted** : non-root, pas d'escalade de privilèges, capacités retirées, seccomp obligatoire.
Manifestes (`k8s/`)	UID fixes, système de fichiers en lecture seule, `drop: [ALL]` , pas de jeton de compte de service, limites mémoire, sondes de santé, image backend désignée par son **digest**.
Kyverno 1.20	Signature Cosign du pipeline exigée ; registres autorisés, tag obligatoire, `latest` interdit, non-root, pas de conteneur `privileged` .
NetworkPolicies	Tout est interdit par défaut, puis seuls sont ouverts : DNS, Traefik → frontend, frontend → backend, backend → base.

7.2 Faille V5 et sa correction

La NetworkPolicy du backend autorisait volontairement la sortie vers `0.0.0.0/0` . Le contrôle IaC du pipeline l'a détectée (règle Trivy KUBE-0002, HIGH) : sur la Pull Request d'introduction du cluster, **seul ce contrôle était rouge** (preuve 93). Le test exécuté sur le cluster réel l'a confirmé : le backend joignait Internet (preuve 94). La correction (PR #20) supprime cette ouverture ; les mises à jour passent par une nouvelle image signée, jamais par le pod.



Figure 15 — V5 détectée par le pipeline : KUBE-0002 sur `network.tf:129` (preuve 93).

7.3 Vérification : tests de segmentation et d'admission

Le script `scripts/k8s_segmentation_test.sh` tente réellement chaque flux interdit et chaque déploiement non conforme.

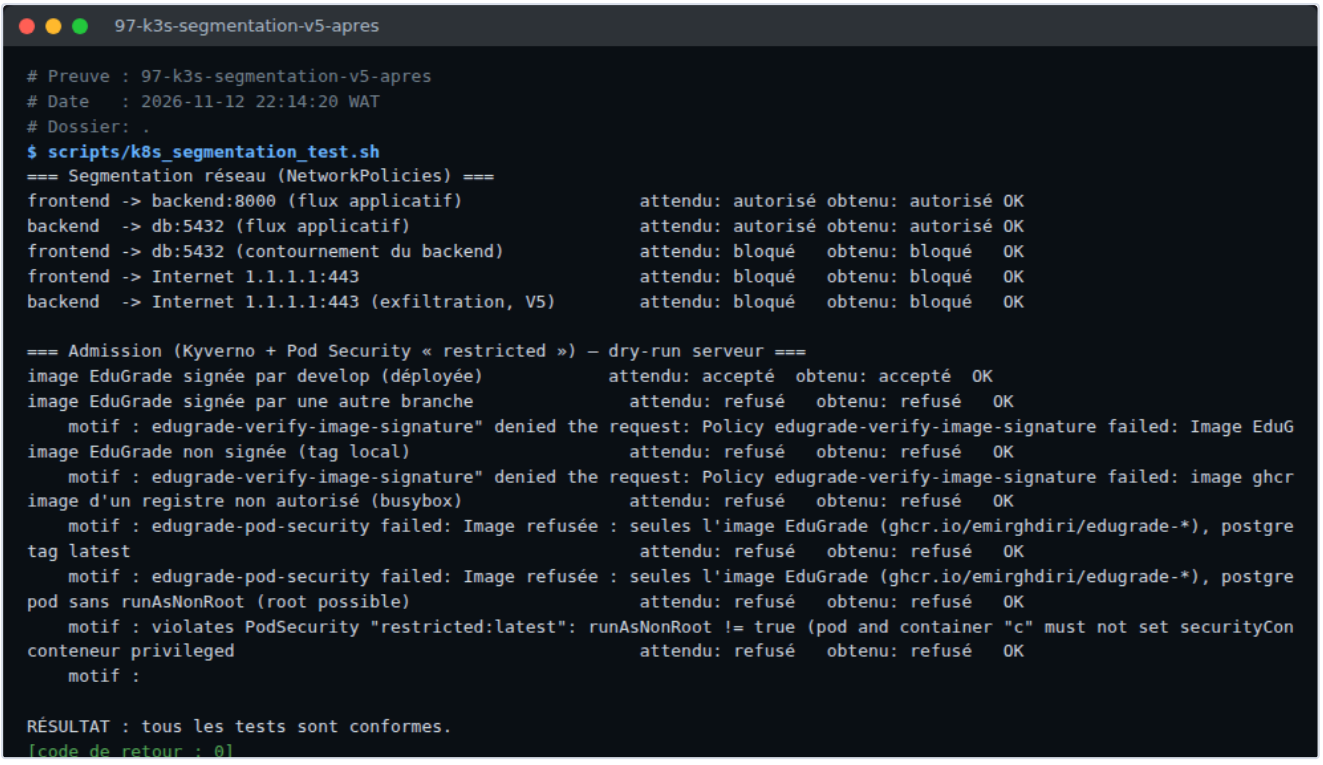


Figure 16 — 12 tests sur 12 conformes : flux autorisés fonctionnels, exfiltration bloquée ; image signée acceptée, et refus d'une image signée par une autre branche, d'une image non signée, d'un registre non autorisé, de `latest`, d'un pod root et d'un conteneur privilégié (preuve 97).

CHAPITRE 8

Observabilité et réponse à incident

> Une protection peut échouer : il faut donc voir les attaques et savoir y répondre.

8.1 Collecte et alertes

Les journaux JSON du backend sont collectés par Grafana Alloy, indexés dans Loki, puis exploités par le tableau de bord « **Sécurité EduGrade** » et trois alertes, toutes déclarées dans Terraform.

Alerte	Déclenchement (sur 5 minutes)	Gravité
Force brute sur la connexion	plus de 5 connexions échouées	critique
Tentative d'injection (SQLi / XSS)	au moins une entrée suspecte	critique
Accès non autorisé (IDOR)	plus de 3 accès refusés	avertissement

8.2 Scénario d'attaque fictive

Le script `scripts/attack_scenario.sh` enchaîne 12 essais de mots de passe, une connexion réussie, quatre injections SQL et XSS, puis l'énumération des notes de six étudiants. **Toutes les attaques échouent** (401, aucune donnée renvoyée, 403) et **toutes sont détectées**.

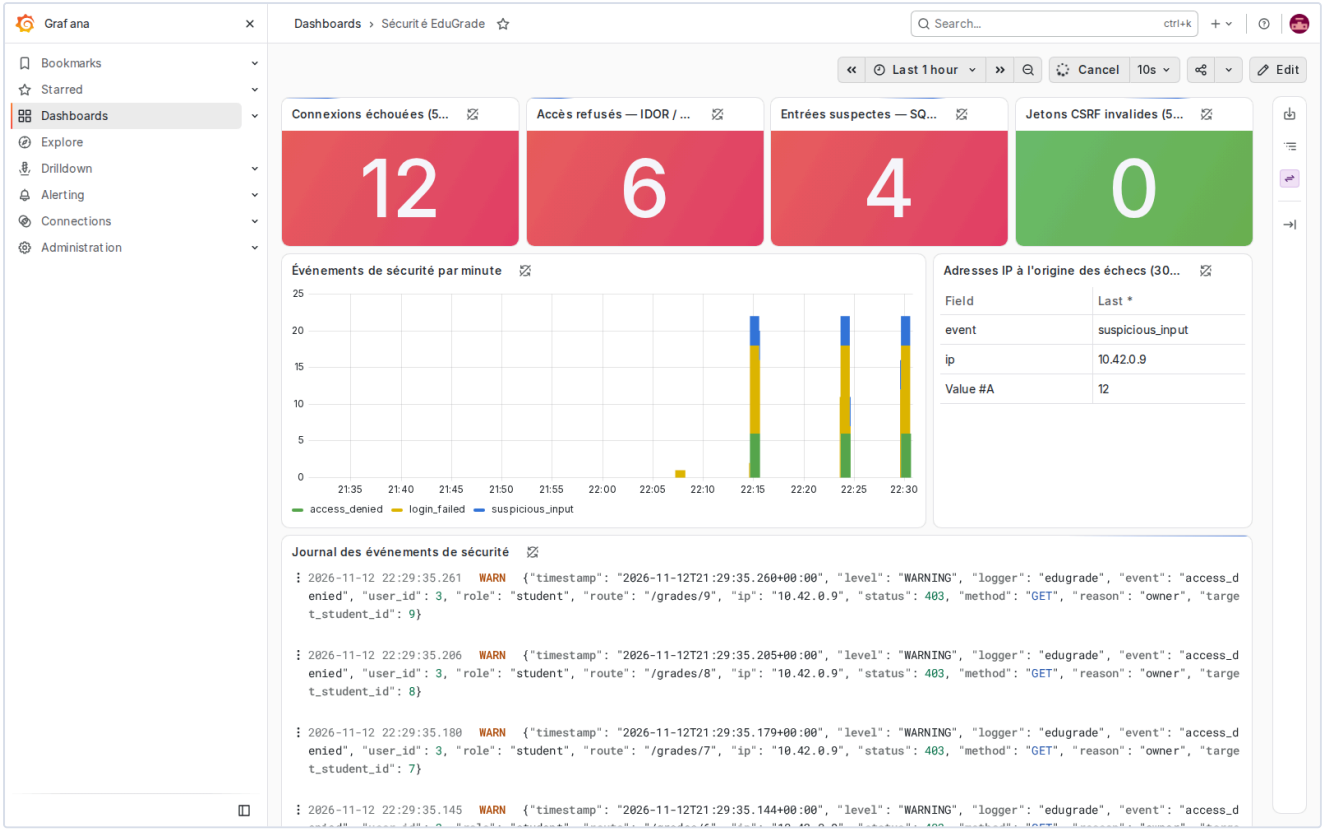


Figure 17 — Tableau de bord pendant l'attaque : 12 connexions échouées, 6 accès refusés, 4 entrées suspectes, vagues d'attaque et journal détaillé (preuve 100).

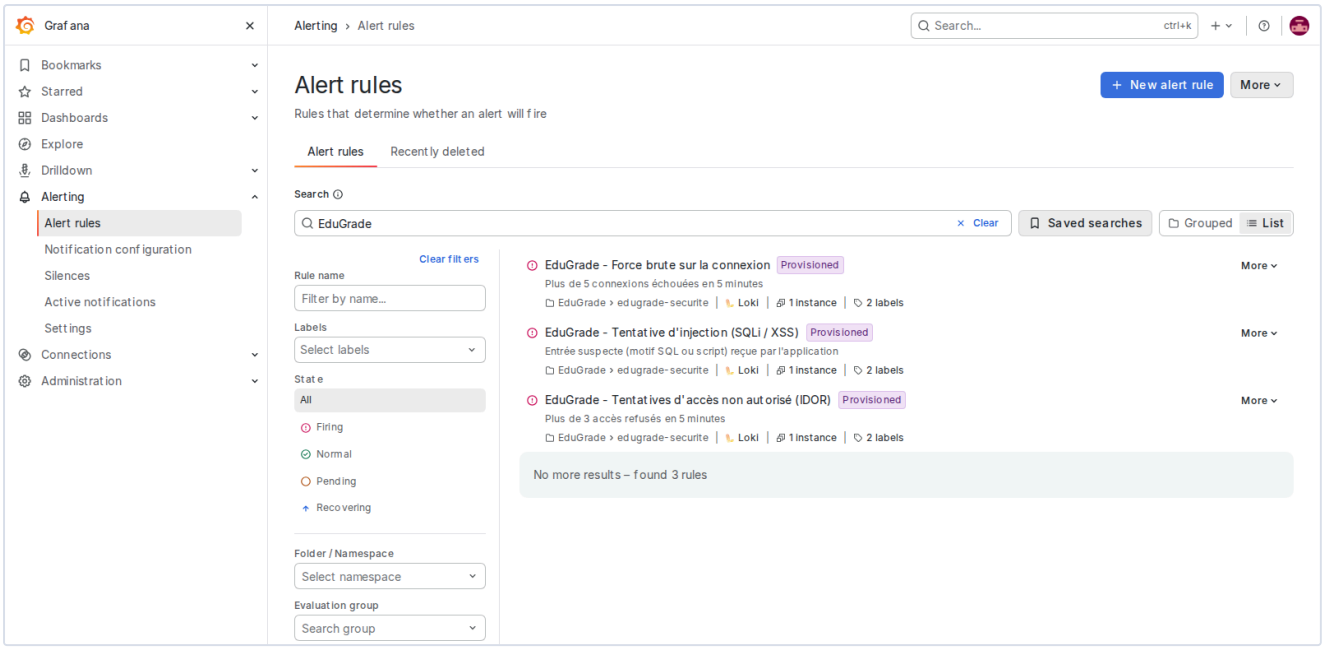


Figure 18 — Les trois règles d'alerte déclenchées (preuve 100).

8.3 Plan de réponse à incident

Le plan (`docs/PLAN_REPONSE_INCIDENT.md`) suit les phases du NIST SP 800-61 :

Phase	Actions principales
Préparation	Journaux centralisés, tableau de bord, alertes, inventaire de l'image déployée (digest, SBOM, signature).
Détection et analyse	Qualification en 15 minutes : comptes et routes visés, requêtes Loki prêtes à l'emploi, classement de gravité.
Confinement	Blocage d'adresse à l'entrée, invalidation de toutes les sessions par rotation de la clé, retour à la dernière image saine signée.
Éradication	Correction soumise à tout le pipeline, test de non-régression reproduisant l'attaque, rotation des secrets exposés.
Rétablissement	Déploiement de l'image corrigée et signée, nouvelle exécution des tests de segmentation et du scénario d'attaque.
Retour d'expérience	Note sous cinq jours, actions correctives datées, mise à jour du plan.

CHAPITRE 9

Analyse des risques (STRIDE)

> Le modèle complet (`docs/STRIDE.md`) recense les actifs (notes, comptes, clé de session, mot de passe de la base, image, journaux) et cinq frontières de confiance : Internet → cluster, proxy → application, application → base, code → pipeline, registre → cluster.

9.1 Menaces et mesures

Catégorie	Menace principale	Mesures en place
Spoofing	Cookie de session forgé (V3), force brute, image usurpant celle du pipeline	Secrets hors du code et rotation ; scrypt et alerte ; signature vérifiée par Kyverno
Tampering	Injection SQL (V2), XSS (B2), YAML malveillant (V4), image modifiée	Requêtes paramétrées, échappement, CSP, `safe_load` ; image par digest signée ; `main` protégée
Repudiation	Nier une action ou une tentative	Journaux centralisés (utilisateur, rôle, route, statut) ; historique des PR ; Rekor
Information disclosure	IDOR (V1), exfiltration (V5), secrets dans les journaux CI	Contrôle d'appartenance ; interdiction réseau par défaut ; sorties d'outils masquées
Denial of service	Épuisement des ressources, indisponibilité non détectée	Limites mémoire et taille de requête ; sondes de santé ; Prometheus
Elevation of privilege	Étudiant → enseignant, évasion de conteneur, pod privilégié	Rôles côté serveur ; non-root et lecture seule ; Pod Security et Kyverno

9.2 Risques résiduels acceptés

Risque	Décision
Dépôt public	Accepté jusqu'au 30/06/2027 : protection de branche gratuite, données fictives.
Commits non signés	Accepté jusqu'au 30/06/2027 : compensé par PR, approbation et CI obligatoires.
Anciennes valeurs V3 dans l'historique	Valeurs révoquées (rotation prouvée) ; la réécriture de l'historique de `main` est interdite.
Pas de limitation de débit sur `/login`	À traiter (middleware Traefik) ; atténué par l'alerte de force brute.

CHAPITRE 10

Difficultés rencontrées et enseignements

> Plusieurs problèmes réels sont apparus pendant la mise en œuvre. Nous les avons traités comme des constats de sécurité : chacun est documenté et prouvé.

Constat	Cause et résolution	Preuves
Secrets affichés par un outil de sécurité	Bandit citait la valeur des mots de passe dans un journal public ; sortie reconfigurée sans valeur.	48, 51, 52
Règles d'admission ignorées	Kyverno 1.20 n'exécute plus l'ancien format de règles ; migration vers le nouveau format (CEL). Sans test d'admission, l'absence de protection serait passée inaperçue.	91, 97
NetworkPolicies non appliquées	Incompatibilité entre l'iptables d'Ubuntu 26.04 et le contrôleur réseau de k3s : le frontend joignait la base. Solution définitive : mode `iptables-legacy` pour tout le système, validé après redémarrage.	92, 95, 105
Limite d'un outil d'analyse	La règle Trivy KUBE-0002 n'examine que la première règle de sortie : placée en second, la même erreur passe inaperçue. Compensé par le test sur le cluster réel.	93, 97
Adresse de l'attaquant introuvable	L'adresse journalisée était celle du proxy interne ; paramètre du nombre de proxys de confiance, testé contre un en-tête falsifié.	102-104
Image de base vulnérable	zlib vulnérable alors qu'un correctif existait : correctifs appliqués à chaque construction.	45, 73, 74
Environnement contraint	Réseau lent et mémoire limitée : charts en cache local, délais adaptés, mémoire WSL portée à 10 Go.	90, 105

Enseignement principal

Un contrôle de sécurité qui n'est pas testé n'existe pas. Deux de nos protections — Kyverno et les NetworkPolicies — étaient déclarées, installées et affichées comme opérationnelles, mais inopérantes. Seuls des tests de comportement, qui tentent réellement de les contourner, l'ont révélé. C'est pourquoi chaque mesure du projet est accompagnée d'un tel test.

CONCLUSION GÉNÉRALE

Conclusion et perspectives

EduGrade démontre une chaîne DevSecOps complète et vérifiable. Les sept failles volontaires ont toutes été **détectées automatiquement**, ont **bloqué la livraison**, puis ont été **corrigées et vérifiées**. L'image déployée est exactement celle qui a été contrôlée : signée par le pipeline, décrite par une SBOM et refusée par le cluster si elle ne l'est pas. À l'exécution, les conteneurs sont non-root et isolés ; une attaque fictive a été bloquée, rendue visible dans Grafana et signalée par trois alertes.

Au-delà des outils, ce projet nous a appris que la sécurité se prouve : plusieurs protections apparemment fonctionnelles étaient inopérantes, et seuls des tests de contournement l'ont montré.

Perspectives

- Limitation de débit sur la page de connexion et verrouillage temporaire de compte.
- Répartiteur de charge conservant l'adresse du client (MetalLB) et chiffrement TLS avec HSTS.
- Signature des commits, et réécriture contrôlée de l'historique pour supprimer les anciens secrets.
- Épinglage des actions GitHub par empreinte et notifications d'alertes vers une messagerie.
- Détection à l'exécution (Falco) et base de données gérée avec sauvegardes chiffrées.

DÉCLARATION

Usage de l'intelligence artificielle

Conformément au cahier des charges, nous déclarons avoir utilisé un assistant d'intelligence artificielle, **Claude Code** (Anthropic), au cours du projet.

Usage	Détail
Code et configuration	Propositions pour le workflow CI/CD, les scripts (ZAP, captures, tests de segmentation, scénario d'attaque), le Terraform du cluster, les règles Kyverno et les corrections des failles.
Diagnostic	Analyse des journaux d'échec et propositions de corrections.
Documentation	Aide à la rédaction de la politique de blocage, du modèle STRIDE, du plan de réponse à incident et du présent rapport.

Contrôle humain

Toutes les propositions ont été relues. Chaque modification est passée par une Pull Request dont la fusion a été décidée par un membre de l'équipe ; l'assistant n'avait pas le droit de fusionner. Les résultats présentés reposent sur les sorties des outils et non sur les affirmations de l'assistant : chaque preuve contient la commande exécutée, sa sortie et son code de retour, et chaque run du pipeline est consultable sur GitHub. Aucune donnée personnelle réelle ni aucun secret réel n'a été transmis à l'assistant.

ANNEXES

Index des preuves et captures complémentaires

Annexe A — Index des preuves

Preuves	Contenu
00 – 06	Prérequis, Terraform du dépôt GitHub, historique git
07 – 18	Détections locales de la Partie 1, Docker Compose, conteneurs non-root, journaux JSON
20 – 34	Application et démonstration des failles avant correction
40 – 52	Premier run du pipeline et correction de l'erreur Bandit
60 – 76	Correction de chaque faille : diff et run de la Pull Request
80 – 86	Pipeline entièrement vert, publication, signature, SBOM, DAST
90 – 97, 101	Cluster k3s, V5 avant et après (CI et cluster), tests d'admission
98 – 100	Scénario d'attaque, détection dans Loki, alertes et tableau de bord
102 – 105	Adresse du client, déploiement de la version finale, configuration réseau définitive

Index détaillés : `docs/PREUVES_PARTIE1.md`, `docs/preuves/corrvulnecicd/INDEX.md`. Documents de référence : `docs/VULNERABILITES.md`, `docs/POLITIQUE_BLOCAGE.md`, `docs/STRIDE.md`, `docs/PLAN_REPONSE_INCIDENT.md`.

Annexe B — Captures complémentaires

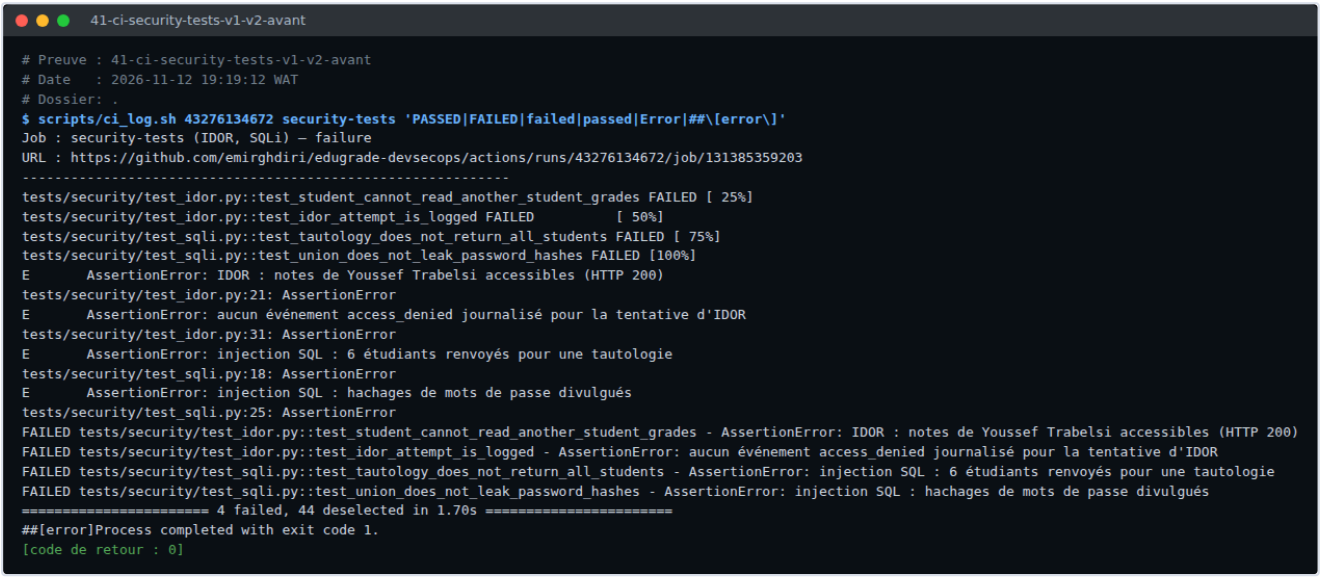


Figure B1 — Tests de sécurité en échec sur la version vulnérable (preuve 41).

43-ci-secrets-gitleaks-v3-avant

```
# Preuve : 43-ci-secrets-gitleaks-v3-avant
# Date   : 2026-11-12 19:19:25 WAT
# Dossier: .
$ scripts/ci_log.sh 43276134672 secrets 'Finding|Secret|RuleID|File|Line|Commit|Author|leaks|##\[error\]'
Job : secrets (gitleaks) — failure
URL : https://github.com/emirghdiri/edugrade-devsecops/actions/runs/43276134672/job/131385359270
------------------------------------------------------------
  --report-format json --report-path /repo/reports/gitleaks.json
  GITLEAKS_IMAGE: ghcr.io/gitleaks/gitleaks:v8.30.1
Unable to find image 'ghcr.io/gitleaks/gitleaks:v8.30.1' locally
v8.30.1: Pulling from gitleaks/gitleaks
Status: Downloaded newer image for ghcr.io/gitleaks/gitleaks:v8.30.1
Finding:     SECRET_KEY = "REDACTED"
Secret:      REDACTED
RuleID:      generic-api-key
File:        app/config.py
Line:        10
Commit:      f78fd006d264ae3feb09585813ffaf312df0a8db
Author:      Amir Ghdiri
Finding:     DB_PASSWORD = "REDACTED"
Secret:      REDACTED
```

Figure B2 — Gitleaks détecte V3 sur tout l'historique, valeurs masquées (preuve 43).

94-k3s-segmentation-v5-avant

```
# Preuve : 94-k3s-segmentation-v5-avant
# Date   : 2026-11-12 22:13:42 WAT
# Dossier: .
$ scripts/k8s_segmentation_test.sh
=== Segmentation réseau (NetworkPolicies) ===
frontend -> backend:8000 (flux applicatif)                attendu: autorisé obtenu: autorisé OK
backend  -> db:5432 (flux applicatif)                     attendu: autorisé obtenu: autorisé OK
frontend -> db:5432 (contournement du backend)            attendu: bloqué   obtenu: bloqué   OK
frontend -> Internet 1.1.1.1:443                          attendu: bloqué   obtenu: bloqué   OK
backend  -> Internet 1.1.1.1:443 (exfiltration, V5)       attendu: bloqué   obtenu: autorisé ÉCHEC

=== Admission (Kyverno + Pod Security « restricted ») — dry-run serveur ===
image EduGrade signée par develop (déployée)           attendu: accepté  obtenu: accepté  OK
image EduGrade signée par une autre branche              attendu: refusé   obtenu: refusé   OK
    motif : edugrade-verify-image-signature" denied the request: Policy edugrade-verify-image-signature failed: Image EduG
image EduGrade non signée (tag local)                    attendu: refusé   obtenu: refusé   OK
    motif : edugrade-verify-image-signature" denied the request: Policy edugrade-verify-image-signature failed: image ghcr
image d'un registre non autorisé (busybox)               attendu: refusé   obtenu: refusé   OK
    motif : edugrade-pod-security failed: Image refusée : seules l'image EduGrade (ghcr.io/emirghdiri/edugrade-*), postgre
tag latest                                               attendu: refusé   obtenu: refusé   OK
    motif : edugrade-pod-security failed: Image refusée : seules l'image EduGrade (ghcr.io/emirghdiri/edugrade-*), postgre
pod sans runAsNonRoot (root possible)                    attendu: refusé   obtenu: refusé   OK
    motif : violates PodSecurity "restricted:latest": runAsNonRoot != true (pod and container "c" must not set securityCon
```

Figure B3 — V5 sur le cluster : le backend joint Internet avant correction (preuve 94).

99-k3s-detection-loki-alertes

```
# Preuve : 99-k3s-detection-loki-alertes
# Date   : 2026-11-12 22:30:22 WAT
# Dossier: .
$ scripts/observability_check.sh
=== Loki : événements de sécurité (namespace edugrade, 15 min) ===
access_denied      12
login_failed       24
login_success      4
suspicious_input   8

=== Loki : exemples (suspicious_input, access_denied) ===
2026-11-12T21:29:35.260+00:00 access_denied user=3 role=student route=/grades/9 status=403
2026-11-12T21:29:35.205+00:00 access_denied user=3 role=student route=/grades/8 status=403
2026-11-12T21:29:35.179+00:00 access_denied user=3 role=student route=/grades/7 status=403
2026-11-12T21:29:35.144+00:00 access_denied user=3 role=student route=/grades/6 status=403
2026-11-12T21:29:35.131+00:00 access_denied user=3 role=student route=/grades/5 status=403
2026-11-12T21:29:35.116+00:00 access_denied user=3 role=student route=/grades/4 status=403

=== Grafana : règles d'alerte EduGrade ===
firing    EduGrade - Force brute sur la connexion
firing    EduGrade - Tentative d'injection (SQLi / XSS)
```

Figure B4 — Événements de l'attaque retrouvés dans Loki et alertes déclenchées (preuve 99).

103-k3s-deploiement-v1.0-secure

```
# Preuve : 103-k3s-deploiement-v1.0-secure
# Date   : 2026-11-12 23:08:04 WAT
# Dossier: .
$ git diff -- k8s/backend.yaml; scripts/k8s_deploy.sh 2>&1 | grep -vE '^Waiting'; echo; kubectl -n edugrade get deploy backend -o jsonpath='{.spec.template.spec.containers
[0].image}{"\n"}'; kubectl -n edugrade exec deploy/backend -- printenv TRUSTED_PROXY_COUNT
diff --git a/k8s/backend.yaml b/k8s/backend.yaml
index ea6100f..f207241 100644
--- a/k8s/backend.yaml
+++ b/k8s/backend.yaml
@@ -34,8 +34,8 @@ spec:
         seccompProfile: { type: RuntimeDefault }
       containers:
         - name: backend
-          # Image du commit a4511f2 (develop), signée par ci.yml@refs/heads/develop.
-          image: ghcr.io/emirghdiri/edugrade-backend@sha256:d3707f8d8e3c6083f6c12f9701997ff3fd688bf58a9d0ddf23a7c6b0e42b69db
+          # Image du commit 82c0eaa (develop), signée par ci.yml@refs/heads/develop.
+          image: ghcr.io/emirghdiri/edugrade-backend@sha256:9992e9c40578b042a0256fbbda1951e3f0b9582744d522972644ed8eb2c7e9b0
           imagePullPolicy: IfNotPresent
           ports:
             - containerPort: 8000
configmap/nginx-conf unchanged
service/backend unchanged
service/db unchanged
service/frontend unchanged
deployment.apps/backend configured
deployment.apps/frontend unchanged
statefulset.apps/db configured
ingress.networking.k8s.io/edugrade unchanged
partitioned roll out complete: 1 new pods have been updated...
deployment "backend" successfully rolled out
deployment "frontend" successfully rolled out
NAME                       READY   STATUS        RESTARTS   AGE   IP          NODE              NOMINATED NODE   READINESS GATES
backend-64cc9b6d5-s2z9r    1/1     Terminating   0          61m   10.42.0.35  desktop-nga3akd   <none>           <none>
backend-7b6dd56d5d-ci69p   1/1     Running       0          18s   10.42.0.40  desktop-nga3akd   <none>           <none>
```

Figure B5 — Déploiement de la version finale : image signée acceptée par Kyverno (preuve 103).